

Project 1: The Mystery Binary

Himadri Saha

EECE.5821 Computer Architecture

10/7/2026

Part C

C.1

This program reads the integer n and prints the sum of the squares.

```
n = read_int()
for i = 1..n: arr[i-1] = mult(i, i)    # arr at 0x10010840
sum = arr[0] + ... + arr[n-1]
print "=> ", sum, '\n' then exit
mult(a, b) multiplies by repeated addition (b loops)
```

C.2

Integer	Float	CSR	Watches
Name	Value	Description	
PC	0x0040008c	Program Counter	
fflags	0x00000000	FP Exception Flags	
frm	0x00000000	FP Rounding Mode	
fcsr	0x00000000	FP Control/Status	
cycle	0x000001b9	Cycle Counter (lo)	
instret	0x000001b9	Instructions Retired (lo)	
misa	0x40001128	ISA Extensions	

ic_n10 = 441

C.3

IC(n) = $\sum s_i \cdot k_i(n)$:

$IC(n) = 6(1) + 1(n+1) + 3n + 5n + 2(1) + 1(n+1) + 6n + 11(1) + 1n + 1[n(n+1)/2 + n] + 3[n(n+1)/2] + 2n$

(B9 and B10):

$(1 + 3) \cdot n(n+1)/2 = 4 \cdot n(n+1)/2 = 2n(n+1) = 2n^2 + 2n$

remaining n terms:

$n(B1) + 3n(B2) + 5n(B3) + n(B5) + 6n(B6) + n(B8) + n(B9) + 2n(B11) = 20n$

Collect the constants:

$$6 (B0) + 1 (B1) + 2 (B4) + 1 (B5) + 11 (B7) = 21$$

Solve:

$$IC(n) = (2n^2 + 2n) + 20n + 21$$

$$IC(n) = 2n^2 + 22n + 21$$

$$a = 2, b = 22, c = 21$$

C.6

- ALU contributes to the most cycles, it's the most common class and has the highest CPI amongst the common classes (4 compared with 3 for branches and jumps)

$$- f = 72\,100 / 78\,206 = 0.922 \rightarrow 92.2\%$$

$$- \text{Speedup} = 1 / [(1 - f) + f/s]$$

As $s \rightarrow \infty$, the f/s term goes to 0:

$$\text{Speedup}_{\text{max}} = 1 / (1 - f) = \text{total cycles} / (\text{cycles not in mult})$$

Numbers:

$$\text{Speedup}_{\text{max}} = 78\,206 / (78\,206 - 72\,100) = 78\,206 / 6\,106 = 12.81\times$$

Part D

# Optimization	IC Changes	CPI Changes	Why it helps
Closed-form sum: $\sum i^2$ 1 = $n(n+1)(2n+1)/6$ replaces both loops	Large drop: $O(n^2) \rightarrow$ constant	Rises (uses 30-cycle mul/div)	Removes all the repeated work. This is an algorithm change, not a small tweak.
No array: squares are 2 never stored and reloaded	Drops by $2n$ (n sw + n lw)	Removes 4- CPI stores and 5-CPI loads	The array was only temporary storage, and memory operations are the most expensive non-MulDiv class.
No subroutine call: 3 mult is gone entirely	Drops by $4n$ (jal, jalr, setup, return move)	—	Removes call overhead along with the loop inside it.

# Optimization	IC Changes	CPI Changes	Why it helps
Strength reduction: 4 $\div 2$ done with srl instead of div	— (still 1 instruction)	Drops: 4 cycles instead of 30	$n(n+1)$ is always even, so a shift gives the exact result. Saves 26 cycles.
$2n + 1$ computed as 5 add t2, a0, t0 , i.e. $n + (n+1)$	Drops by 1 (instead of slli + addi)	—	Reuses the $n+1$ already computed. Saves 4 cycles.
Removed the 6 redundant addi a0, a0, 0 after lui	Drops by 1	—	The string is at 0x10010000, so its low 12 bits are 0 and lui alone builds the full address. Saves 4 cycles.
Reordered to 7 compute the result straight into a0	Drops by 1 (no mv before print-int)	—	The work starts while a0 still holds n , so the final divu writes the value to print directly into a0. Saves 4 cycles.
8 Negative-input guard (bge)	Rises by 1	—	Costs 3 cycles. Kept for correctness: the original prints 0 for $n < 1$, but the formula alone would print nonzero values for some negative n (for example, $n = -2$ gives -1).

RV32M is worth it for any meaningful n . Each 30-cycle instruction replaces work that grows with n , so its fixed cost pays for itself quickly. For very small inputs, though, a pure RV32I version would be faster.

When mul costs 30 cycles:

i	Repeated addition ($14i + 14$)	mul (30)	Winner
1	28	30	repeated addition
2	42	30	mul
10	154	30	mul ($5\times$)
100	1 414	30	mul ($47\times$)

Without RV32M, the best approach is an $O(n)$ loop that builds each square from the previous one with no multiply:

$$(i+1)^2 = i^2 + (2i + 1)$$

Total $\approx 85 + 19n$ cycles.

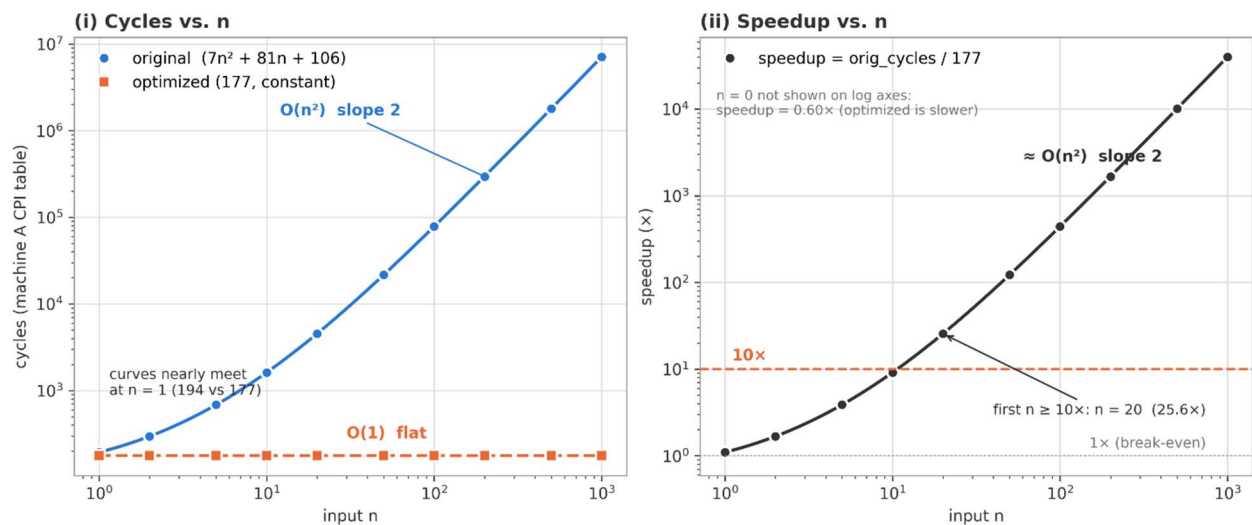
For which n is the optimized version not much faster?

n	Original cycles ($7n^2 + 81n + 106$)	Optimized	Speedup
0	106	177	0.60× (slower)
1	194	177	1.10×
2	296	177	1.67×
5	686	177	3.88×
10	1 616	177	9.13×
20	4 526	177	25.6×
100	78 206	177	441.8×

At $n \leq 2$, optimized version is faster. At $n = 0$, the version is slower. This is because the original version cost grows with n , but the optimized cost is fixed. At smaller values for n , the original n^2 is ~ 0 , meaning it costs a little more than setup and printing instructions.

Graduate component

G3.



The optimization lowers IC but raises CPI. The optimized code runs only 19 instructions, but 3 of them are mul/mul/divu at 30 cycles each. Those 3 take 90 of its 177 cycles. The optimized program is slower at $n = 0$. At $n = 0$ the original takes 106 cycles and the optimized takes 177.

G2.

No, machine B is faster on the original program but slower on the optimized program. Which machine wins depends on the instructions used.

Instruction mix of each program:

Class	Original: count (% of IC)	Optimized: count (% of IC)
ALU	11 215 (50.5%)	11 (57.9%)
Branch + Jump	10 802 (48.6%)	1 (5.3%)
Load + Store	200 (0.9%)	0
MulDiv	0	3 (15.8%)
System	4 (0.02%)	4 (21.1%)
Total	22 221	19

Original Program - B is faster: B's large ALU advantage outweighs the rest of the instructions, and B finishes in 110.7 μ s against A's 156.4 μ s

Optimized Program – A is faster: B still saves time on ALU and MulDiv instructions, but there are only 14 of those. This can't make up for the system calls plus B's slower clock.

A speedup is the property of a machine AND workload together (not the machine alone)